

PROBLEM 01 • Move Zeroes

EASY | Two Pointer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Rearrange an array in-place so all zeroes move to the end while preserving non-zero order.

2. CORE OBSERVATION

Observation

Keep a write pointer for the next non-zero; the array before write contains exactly the non-zero values seen so far.

STATE

write = next position to fill; scan index i is the processed boundary.

INVARIANT

After processing i, nums[0..write-1] are exactly the non-zero elements from nums[0..i] in original order.

TRANSITION

When nums[i] != 0, copy/swap it to write and increment write; otherwise only advance i.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Two Pointer
Trigger: identify the signal in the question.
Memory sentence: Keep a write pointer for the next non-zero; the array before write contains exactly the non-zero values seen so far.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 02 • Two Sum II

MEDIUM | Two Pointer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find two indices in a sorted array whose values sum to target.

2. CORE OBSERVATION

Observation

Sorted order lets us discard an entire side after comparing the sum with target.

STATE

INVARIANT

TRANSITION

l and r are candidate endpoints.

All discarded pairs cannot be a solution.

If sum < target l++; if sum > target r--;
else return the pair.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Two Pointer

Trigger: identify the signal in the question.

Memory sentence: Sorted order lets us discard an entire side after comparing the sum with target.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 03 • 3Sum

MEDIUM | Two Pointer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find all unique triplets whose sum is zero.

2. CORE OBSERVATION

Observation

Sort first; fix one value, then solve a Two-Sum problem with two pointers and skip duplicates.

STATE

INVARIANT

TRANSITION

i is the fixed element; l/r bound the remaining candidates.

For a fixed i, every untested pair lies within [l,r].

Move l/r by sum comparison and skip equal values after recording a triplet.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Two Pointer
Trigger: identify the signal in the question.
Memory sentence: Sort first; fix one value, then solve a Two-Sum problem with two pointers and skip duplicates.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 04 • Sort Colors

MEDIUM | Two Pointer / Dutch National Flag

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Sort an array containing only 0,1,2 in one pass and $O(1)$ space.
--------	---

2. CORE OBSERVATION

Observation	Maintain three regions: 0s, unknowns, and 2s.
-------------	---

STATE	INVARIANT	TRANSITION
low = next 0 slot, mid = current unknown, high = next 2 slot.	$[0, low)$ are 0, $[low, mid)$ are 1, $[high+1, n)$ are 2.	0: swap low/mid and advance both; 1: mid++; 2: swap mid/high and decrement high without advancing mid.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL

Pattern: Two Pointer / Dutch National Flag
Trigger: identify the signal in the question.
Memory sentence: Maintain three regions: 0s, unknowns, and 2s.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 05 • Container With Most Water

MEDIUM | Two Pointer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Choose two lines maximizing contained water.

2. CORE OBSERVATION

Observation

Width shrinks every step, so the shorter boundary must move; moving the taller one cannot improve the limiting height.

STATE

INVARIANT

TRANSITION

l/r are current boundaries; best stores maximum area.

Any pair with current shorter line and a narrower width cannot beat the current area unless the shorter height increases.

Compute area; move the shorter side inward.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Two Pointer

Trigger: identify the signal in the question.

Memory sentence: Width shrinks every step, so the shorter boundary must move; moving the taller one cannot improve the limiting height.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 06 • Trapping Rain Water

HARD | Two Pointer / Prefix reasoning

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Compute total water trapped between bars.

2. CORE OBSERVATION

Observation

At each position water depends on $\min(\text{maxLeft}, \text{maxRight}) - \text{height}$.

STATE

INVARIANT

TRANSITION

Use l/r plus $\text{leftMax}/\text{rightMax}$.

$\text{leftMax}/\text{rightMax}$ represent the highest wall seen on each side of the current boundary.

Process the side with smaller boundary height; update its max and add $\text{max} - \text{height}$.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Two Pointer / Prefix reasoning

Trigger: identify the signal in the question.

Memory sentence: At each position water depends on $\min(\text{maxLeft}, \text{maxRight}) - \text{height}$.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 07 • Maximum Sum Subarray of Size K

EASY | Fixed Sliding Window

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find the maximum sum among all contiguous windows of fixed size k.

2. CORE OBSERVATION

Observation

Adjacent windows differ by one outgoing and one incoming element.

STATE

INVARIANT

TRANSITION

windowSum is the sum of the current k elements.

After each shift, windowSum equals exactly the sum of the current window.

Add incoming element, subtract outgoing element, update maximum.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Fixed Sliding Window
Trigger: identify the signal in the question.
Memory sentence: Adjacent windows differ by one outgoing and one incoming element.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 08 • Max Consecutive ones

EASY | Sliding Window

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find the longest contiguous segment containing only 1s.

2. CORE OBSERVATION

Observation

A window is valid exactly while it contains no zero.

STATE

INVARIANT

TRANSITION

l/r define the current valid window.

Window [l,r] contains only ones.

Expand r; when a zero appears, move l past the zero; update max length.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Sliding Window
Trigger: identify the signal in the question.
Memory sentence: A window is valid exactly while it contains no zero.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 09 • Max Consecutive ones III

MEDIUM | Variable Sliding Window

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find longest subarray containing at most k zeroes.

2. CORE OBSERVATION

Observation

Allow bad elements up to a budget; shrink only when budget is exceeded.

STATE

INVARIANT

TRANSITION

zeros = number of zeroes in current window.

zeros $\leq$ k for every maintained window.

Expand r; if zeros > k, move l and remove zero counts; maximize r-l+1.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Variable Sliding Window
Trigger: identify the signal in the question.
Memory sentence: Allow bad elements up to a budget; shrink only when budget is exceeded.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 10 • Subarray Product Less Than K

MEDIUM | Variable Sliding Window

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Count contiguous subarrays whose product is less than k for positive numbers.

2. CORE OBSERVATION

Observation

With positive values, expanding multiplies product; once product $\geq k$, shrink from left.

STATE

INVARIANT

TRANSITION

product is current window product.

Current window product $< k$; every suffix ending at r inside it is also valid.

Multiply $\text{nums}[r]$; shrink while $\text{product} \geq k$; add $r-l+1$.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Variable Sliding Window
Trigger: identify the signal in the question.
Memory sentence: With positive values, expanding multiplies product; once product $\geq k$, shrink from left.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 11 • Fruits Into Baskets

MEDIUM | Variable Sliding Window

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find the longest subarray containing at most two distinct values.

2. CORE OBSERVATION

Observation

It is a standard variable window with a two-type constraint.

STATE

INVARIANT

TRANSITION

frequency map tracks types in $[l, r]$.

Window contains at most two distinct fruit types.

Expand r ; while $\text{distinct} > 2$, decrement/remove $\text{nums}[l]$ and advance l ; update max .

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Variable Sliding Window
Trigger: identify the signal in the question.
Memory sentence: It is a standard variable window with a two-type constraint.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 12 • Minimum Size Subarray Sum

MEDIUM | Variable Sliding Window

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find the minimum length subarray with sum at least target for positive numbers.

2. CORE OBSERVATION

Observation

Positive numbers make the sum monotonic under expansion/shrinkage.

STATE

INVARIANT

TRANSITION

sum is current window sum.

Current window is the smallest valid window ending at r after shrinking.

Expand r; while sum $\geq$ target, record length and remove nums[l].

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Variable Sliding Window
Trigger: identify the signal in the question.
Memory sentence: Positive numbers make the sum monotonic under expansion/shrinkage.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 13 • Sliding Window Maximum

HARD | Monotonic Deque

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Return the maximum of every fixed-size window.

2. CORE OBSERVATION

Observation

A decreasing deque keeps only candidates that can still become the maximum.

STATE

Deque stores indices in decreasing value order.

INVARIANT

Front is always the maximum index in the current window; dominated indices are removed.

TRANSITION

Remove out-of-window front; pop smaller values from back; push r ; once window reaches k , output front.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Monotonic Deque
Trigger: identify the signal in the question.
Memory sentence: A decreasing deque keeps only candidates that can still become the maximum.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 14 • Subarray with k distinct integers

HARD | Sliding Window + Frequency Map

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Count subarrays containing exactly k distinct integers.
--------	---

2. CORE OBSERVATION

Observation	Exactly $k = \text{atMost}(k) - \text{atMost}(k-1)$.
-------------	---

STATE	INVARIANT	TRANSITION
Frequency map and left boundary define an at-most-k window.	For $\text{atMost}(k)$, every subarray ending at r from $l..r$ is valid.	Compute atMost twice and subtract.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL	COMPLEXITY
Pattern: Sliding Window + Frequency Map Trigger: identify the signal in the question. Memory sentence: Exactly $k = \text{atMost}(k) - \text{atMost}(k-1)$.	Time: _____ Space: _____
REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐	Confidence: 1 2 3 4 5

PROBLEM 15 • Find Pivot Index

EASY | Prefix Sum

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find an index where left sum equals right sum.

2. CORE OBSERVATION

Observation

Right sum can be derived from total sum minus left sum and current value.

STATE

INVARIANT

TRANSITION

left is sum before i; total is whole-array sum.

At i, right = total-left-nums[i].

Check equality, then add nums[i] to left.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Prefix Sum

Trigger: identify the signal in the question.

Memory sentence: Right sum can be derived from total sum minus left sum and current value.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 16 • Subarray Sum Equals K

MEDIUM | Prefix Sum + HashMap

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Count subarrays whose sum equals k.
--------	-------------------------------------

2. CORE OBSERVATION

Observation	If prefixSum-k was seen before, each occurrence forms a valid subarray ending here.
-------------	---

STATE	INVARIANT	TRANSITION
prefixSum and frequency map of prior prefix sums.	Map stores how many times each prefix sum has occurred before current index.	Update prefixSum; add freq[prefixSum-k] to answer; then increment freq[prefixSum].

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL

Pattern: Prefix Sum + HashMap
Trigger: identify the signal in the question.
Memory sentence: If prefixSum-k was seen before, each occurrence forms a valid subarray ending here.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 17 • Matrix Block Sum (Running Sum of 2D Array)

MEDIUM | 2D Prefix Sum

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Compute each cell's sum over a k-radius rectangular block.
--------	--

2. CORE OBSERVATION

Observation	A 2D prefix sum answers any rectangle in $O(1)$.
-------------	---

STATE

$P[r][c]$ stores sum of rectangle from origin to (r,c) .

INVARIANT

P encodes all rectangle sums so inclusion-exclusion gives any query rectangle.

TRANSITION

Build prefix; clamp rectangle bounds; use four-corner inclusion-exclusion.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: 2D Prefix Sum
Trigger: identify the signal in the question.
Memory sentence: A 2D prefix sum answers any rectangle in $O(1)$.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 18 • Product of Array Except Self

MEDIUM | Prefix Product

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

For each index, return product of all other elements without division.

2. CORE OBSERVATION

Observation

Answer = product of prefix elements × product of suffix elements.

STATE

First pass stores prefix product in ans;
second pass carries suffix product.

INVARIANT

Before i, ans[i] equals product of elements
left of i; after second pass it includes right
product too.

TRANSITION

Forward multiply prefix; backward
multiply suffix into ans.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables
mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Prefix Product
Trigger: identify the signal in the question.
Memory sentence: Answer = product of prefix elements × product of suffix elements.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 19 • Continuous Subarray Sum

MEDIUM | Prefix Sum + Modulo

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Determine whether a length ≥ 2 subarray sum is a multiple of k .

2. CORE OBSERVATION

Observation

Equal prefix remainders imply the difference is divisible by k .

STATE

Store earliest index for each $\text{prefixSum} \% k$.

INVARIANT

The earliest index for a remainder gives the longest candidate; repeated remainder creates a divisible sum.

TRANSITION

Track remainder; if seen and distance ≥ 2 return true; otherwise store first index.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Prefix Sum + Modulo
Trigger: identify the signal in the question.
Memory sentence: Equal prefix remainders imply the difference is divisible by k .

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 20 • Subarray Sum Divisible by K

MEDIUM | Prefix Sum + Modulo

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Count subarrays whose sum is divisible by k.

2. CORE OBSERVATION

Observation

Equal prefix remainders define divisible subarrays.

STATE

INVARIANT

TRANSITION

remainder frequency map.

freq[r] counts previous prefixes with the same normalized remainder.

Normalize remainder; add freq[r] to answer; increment freq[r].

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Prefix Sum + Modulo
Trigger: identify the signal in the question.
Memory sentence: Equal prefix remainders define divisible subarrays.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 21 • Maximum Subarray

MEDIUM | Kadane / DP

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer Find maximum-sum contiguous subarray.

2. CORE OBSERVATION

Observation At each element, either extend the current best ending here or start fresh here.

STATE

cur = best sum ending at current index;
best = global maximum.

INVARIANT

cur is the maximum sum of a subarray
ending at i.

TRANSITION

cur = max(x, cur + x); best = max(best, cur).

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Kadane / DP
Trigger: identify the signal in the question.
Memory sentence: At each element, either extend the current best ending here or start fresh here.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 22 • Maximum Product Subarray

MEDIUM | Kadane Variant

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find maximum product of a contiguous subarray.

2. CORE OBSERVATION

Observation

A negative can turn the smallest product into the largest, so track both max and min ending here.

STATE

INVARIANT

TRANSITION

curMax and curMin.

curMax/curMin are maximum/minimum products of subarrays ending at i.

For x,
newMax=max(x,x*oldMax,x*oldMin);
newMin=min(...).

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Kadane Variant

Trigger: identify the signal in the question.

Memory sentence: A negative can turn the smallest product into the largest, so track both max and min ending here.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 23 • Maximum Sum Circular Subarray

MEDIUM | Kadane + Circular

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find maximum subarray sum when the array is circular.

2. CORE OBSERVATION

Observation

Answer is $\max(\text{normal Kadane}, \text{total} - \text{minimum subarray})$, except all-negative arrays.

STATE

maxKadane, minKadane, total.

INVARIANT

A wrapping maximum is total minus the middle minimum segment.

TRANSITION

Compute both Kadane values; if $\text{maxKadane} < 0$ return it, else $\max(\text{maxKadane}, \text{total} - \text{minKadane})$.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Kadane + Circular
Trigger: identify the signal in the question.
Memory sentence: Answer is $\max(\text{normal Kadane}, \text{total} - \text{minimum subarray})$, except all-negative arrays.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 24 • Maximum Absolute Sum of Any Subarray

MEDIUM | Kadane on max/min

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find maximum absolute value of any contiguous subarray sum.

2. CORE OBSERVATION

Observation

Max absolute sum = $\max(\text{max subarray sum}, -\text{min subarray sum})$.

STATE

INVARIANT

TRANSITION

maxEnd/minEnd and global max/min sums.

Track extreme positive and negative subarray sums ending at each index.

Run max-Kadane and min-Kadane; return $\max(\text{maxSum}, -\text{minSum})$.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Kadane on max/min
Trigger: identify the signal in the question.
Memory sentence: Max absolute sum = $\max(\text{max subarray sum}, -\text{min subarray sum})$.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 25 • Largest Sum Contiguous Subarray

MEDIUM | Kadane

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find the largest sum over all contiguous subarrays.

2. CORE OBSERVATION

Observation

Same Kadane invariant: best subarray ending here either extends or restarts.

STATE

INVARIANT

TRANSITION

cur and best.

cur is best sum ending at current position.

cur = max(a[i], cur + a[i]);
best = max(best, cur).

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Kadane

Trigger: identify the signal in the question.

Memory sentence: Same Kadane invariant: best subarray ending here either extends or restarts.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 26 • Reverse a String

EASY | Two Pointer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Reverse characters in-place.

2. CORE OBSERVATION

Observation

Swapping symmetric positions is enough.

STATE

INVARIANT

TRANSITION

l/r point to the unmatched ends.

Everything outside [l,r] is already in final reversed position.

Swap s[l],s[r]; l++; r--.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Two Pointer
Trigger: identify the signal in the question.
Memory sentence: Swapping symmetric positions is enough.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 27 • Valid Palindrome

EASY | Two Pointer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Check whether a string is a palindrome ignoring non-alphanumeric characters and case.

2. CORE OBSERVATION

Observation

Compare normalized characters from both ends.

STATE

INVARIANT

TRANSITION

l/r are next characters to compare.

All characters outside [l,r] have already matched.

Skip invalid chars; compare lowercase forms; move both inward.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Two Pointer
Trigger: identify the signal in the question.
Memory sentence: Compare normalized characters from both ends.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 28 • Valid Palindrome II

EASY | Two Pointer + Greedy

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Check if a string can become a palindrome after deleting at most one character.

2. CORE OBSERVATION

Observation

The first mismatch forces the only possible deletion to be one of the mismatched characters.

STATE

INVARIANT

TRANSITION

l/r plus one optional deletion decision.

Before the first mismatch, all compared pairs match; after mismatch test both possible skips.

On mismatch, return `isPal(l+1,r)` or `isPal(l,r-1)`.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Two Pointer + Greedy

Trigger: identify the signal in the question.

Memory sentence: The first mismatch forces the only possible deletion to be one of the mismatched characters.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 29 • Longest Palindromic Substring

MEDIUM | Expand Around Center

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Find the longest palindromic substring.
--------	---

2. CORE OBSERVATION

Observation	Every palindrome has a center; expand around each possible center.
-------------	--

STATE	INVARIANT	TRANSITION
center and expansion boundaries.	At each center, the expanded interval is a palindrome.	Expand for odd and even centers; keep longest interval.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL	COMPLEXITY
Pattern: Expand Around Center Trigger: identify the signal in the question. Memory sentence: Every palindrome has a center; expand around each possible center.	Time: _____ Space: _____
REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐	Confidence: 1 2 3 4 5

PROBLEM 30 • Palindromic Substrings

MEDIUM | Expand Around Center

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Count all palindromic substrings.
--------	-----------------------------------

2. CORE OBSERVATION

Observation	Each palindrome is discovered by expanding around its center.
-------------	---

STATE	INVARIANT	TRANSITION
l/r around each odd/even center.	Every expansion that matches is one distinct palindromic substring.	Expand from every center; increment count on each match.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL	COMPLEXITY
Pattern: Expand Around Center Trigger: identify the signal in the question. Memory sentence: Each palindrome is discovered by expanding around its center.	Time: _____ Space: _____
REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐	Confidence: 1 2 3 4 5

PROBLEM 31 • Find All Anagrams in a String

MEDIUM | Sliding Window + Frequency

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find start indices of permutations of pattern in text.

2. CORE OBSERVATION

Observation

A fixed-size window is an anagram when its character counts match the pattern.

STATE

INVARIANT

TRANSITION

frequency arrays/map for pattern and current window.

Window length equals pattern length and counts match.

Add right char, remove left char when oversized, compare counts.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Sliding Window + Frequency

Trigger: identify the signal in the question.

Memory sentence: A fixed-size window is an anagram when its character counts match the pattern.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 32 • Longest Substring Without Repeating Characters

MEDIUM | Sliding Window + Set/Map

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Find longest substring with no repeated characters.
--------	---

2. CORE OBSERVATION

Observation	When a duplicate appears, move left past its previous occurrence.
-------------	---

STATE

l and lastSeen/visited state.

INVARIANT

Window [l,r] contains unique characters.

TRANSITION

If lastSeen[ch]>=l, set l=lastSeen[ch]+1; update lastSeen and max.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Sliding Window + Set/Map
Trigger: identify the signal in the question.
Memory sentence: When a duplicate appears, move left past its previous occurrence.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 33 • Longest Substring with K Uniques

MEDIUM | Sliding Window + Frequency

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find longest substring containing exactly k distinct characters.

2. CORE OBSERVATION

Observation

Maintain an at-most-k window and subtract at-most-k-1, or directly shrink to k.

STATE

INVARIANT

TRANSITION

frequency map and l/r.

Current window has at most k distinct characters; for exactly k, record only when distinct==k.

Expand; shrink while distinct>k; update when distinct==k.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Sliding Window + Frequency

Trigger: identify the signal in the question.

Memory sentence: Maintain an at-most-k window and subtract at-most-k-1, or directly shrink to k.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 34 • Permutation in String

MEDIUM | Fixed Sliding Window + Frequency

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Check whether s2 contains a permutation of s1.

2. CORE OBSERVATION

Observation

A permutation is exactly a fixed-size window with identical character frequencies.

STATE

INVARIANT

TRANSITION

window frequency counts.

Window size is $|s1|$ and count vector equals s1's count vector.

Slide a fixed window and compare/update counts.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Fixed Sliding Window + Frequency
Trigger: identify the signal in the question.
Memory sentence: A permutation is exactly a fixed-size window with identical character frequencies.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 35 • Minimum Window Substring

HARD | Sliding Window + Frequency

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find the smallest substring of s containing all characters of t with multiplicity.

2. CORE OBSERVATION

Observation

Expand until valid, then shrink greedily while preserving validity.

STATE

INVARIANT

TRANSITION

need counts, window counts, formed/remaining count.

Window is valid when all required character counts are satisfied.

Expand r; once valid, shrink l while valid and update minimum.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Sliding Window + Frequency
Trigger: identify the signal in the question.
Memory sentence: Expand until valid, then shrink greedily while preserving validity.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 36 • Substring with Concatenation of All Words

HARD | Sliding Window + HashMap

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find substrings formed by every word exactly once in any order.

2. CORE OBSERVATION

Observation

All words have equal length, so inspect windows aligned by word length and track word frequencies.

STATE

INVARIANT

TRANSITION

left/right word boundaries and frequency map.

Current aligned window contains no word too many times and uses a fixed number of words.

For each offset, add one word; shrink when a word exceeds target; record when count==number of words.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Sliding Window + HashMap
Trigger: identify the signal in the question.
Memory sentence: All words have equal length, so inspect windows aligned by word length and track word frequencies.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 37 • Binary Search

EASY | Binary Search

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find a target in a sorted array in $O(\log n)$.

2. CORE OBSERVATION

Observation

Sorted order lets us discard half the search space after each comparison.

STATE

INVARIANT

TRANSITION

l/r define remaining candidate interval.

If target exists, it remains inside $[l, r]$.

$mid = (l+r)/2$; compare target and move one boundary.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search

Trigger: identify the signal in the question.

Memory sentence: Sorted order lets us discard half the search space after each comparison.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 38 • Sqrt(x)

EASY | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Return floor(sqrt(x)) without floating-point square root.
--------	---

2. CORE OBSERVATION

Observation	x is monotonic: if $mid * mid \leq x$, mid is feasible and we can search right.
-------------	--

STATE	INVARIANT	TRANSITION
l/r search answer space; ans is best feasible value.	All values $\leq$ ans are feasible; values above current right are untested/infeasible as determined.	Binary search 0..x (or 1..x); update ans when $square \leq x$.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL	COMPLEXITY
Pattern: Binary Search on Answer Trigger: identify the signal in the question. Memory sentence: x is monotonic: if $mid * mid \leq x$, mid is feasible and we can search right.	Time: _____ Space: _____
REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐	Confidence: 1 2 3 4 5

PROBLEM 39 • Search Insert Position

EASY | Binary Search

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find the index where target exists or should be inserted in sorted order.

2. CORE OBSERVATION

Observation

Find the first position whose value is $\geq$ target.

STATE

INVARIANT

TRANSITION

l/r search lower bound.

All indices before l have values $<$ target.

If $\text{nums}[\text{mid}] \geq \text{target}$, move $r = \text{mid} - 1$;
else $l = \text{mid} + 1$; return l.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search

Trigger: identify the signal in the question.

Memory sentence: Find the first position whose value is $\geq$ target.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 40 • Search in Rotated Sorted Array

MEDIUM | Binary Search on Rotated Array

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find target in a sorted array rotated once.

2. CORE OBSERVATION

Observation

At least one half around mid is sorted; determine which and decide whether target lies there.

STATE

INVARIANT

TRANSITION

l/r candidate interval.

Target, if present, remains inside the chosen half after each step.

Identify sorted half; if target falls in its range search it, otherwise discard it.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Binary Search on Rotated Array
Trigger: identify the signal in the question.
Memory sentence: At least one half around mid is sorted; determine which and decide whether target lies there.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 41 • Find Minimum in Rotated Sorted Array

MEDIUM | Binary Search on Rotated Array

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find minimum element in a rotated sorted array.

2. CORE OBSERVATION

Observation

Compare mid with right: if $\text{nums}[\text{mid}] > \text{nums}[\text{r}]$, minimum lies right; otherwise it is at mid or left.

STATE

INVARIANT

TRANSITION

l/r candidate interval.

Minimum always remains within $[l, r]$.

Use comparison with right to discard half until $l == r$.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Binary Search on Rotated Array
Trigger: identify the signal in the question.
Memory sentence: Compare mid with right: if $\text{nums}[\text{mid}] > \text{nums}[\text{r}]$, minimum lies right; otherwise it is at mid or left.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 42 • Find Peak Element

MEDIUM | Binary Search

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find any index whose value is greater than its neighbors.

2. CORE OBSERVATION

Observation

If $\text{nums}[\text{mid}] < \text{nums}[\text{mid}+1]$, a peak must exist to the right; otherwise one exists at mid/left .

STATE

INVARIANT

TRANSITION

l/r candidate range.

There is guaranteed to be a peak inside the current interval.

Compare mid with $\text{mid}+1$ and move toward the rising side.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search

Trigger: identify the signal in the question.

Memory sentence: If $\text{nums}[\text{mid}] < \text{nums}[\text{mid}+1]$, a peak must exist to the right; otherwise one exists at mid/left .

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 43 • Find kth rotation

EASY | Binary Search / Rotation Count

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find how many positions a sorted array was rotated.

2. CORE OBSERVATION

Observation

The rotation count is the index of the minimum element.

STATE

INVARIANT

TRANSITION

l/r search for minimum; index of min is k.

Minimum stays inside the candidate interval.

Use rotated-array minimum search; return min index.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search / Rotation Count
Trigger: identify the signal in the question.
Memory sentence: The rotation count is the index of the minimum element.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 44 • Count Occurrences

EASY | Binary Search Bounds

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Count occurrences of a target in a sorted array.

2. CORE OBSERVATION

Observation

Frequency = lastOccurrence - firstOccurrence + 1.

STATE

INVARIANT

TRANSITION

Two binary searches for lower/upper bound.

Lower bound is first $\geq$ target; upper bound is first $>$ target.

Find both boundaries and subtract; return 0 if absent.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search Bounds
Trigger: identify the signal in the question.
Memory sentence: Frequency = lastOccurrence - firstOccurrence + 1.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 45 • Ceiling in a Sorted Array

EASY | Binary Search Bound

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Find the smallest element $\geq$ target.
--------	--

2. CORE OBSERVATION

Observation	This is lower bound.
-------------	----------------------

STATE

l/r and answer index.

INVARIANT

All indices before answer are $<$ target.

TRANSITION

If $\text{nums}[\text{mid}] \geq \text{target}$ record mid and move left; else move right.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Binary Search Bound
Trigger: identify the signal in the question.
Memory sentence: This is lower bound.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 46 • Floor in a Sorted Array

EASY | Binary Search Bound

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Find the largest element $\leq$ target.
--------	---

2. CORE OBSERVATION

Observation	This is upper bound minus one.
-------------	--------------------------------

STATE

l/r and answer index.

INVARIANT

All indices after answer are $>$ target.

TRANSITION

If $\text{nums}[\text{mid}] \leq \text{target}$ record mid and move right; else move left.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Binary Search Bound
Trigger: identify the signal in the question.
Memory sentence: This is upper bound minus one.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 47 • Find First and Last Position of Element

MEDIUM | Binary Search Bounds

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Return first and last index of a target in sorted array.

2. CORE OBSERVATION

Observation

Two boundary searches isolate the equal-value range.

STATE

INVARIANT

TRANSITION

lowerBound and upperBound-1.

First is first $\geq$ target; last is first $>$ target minus one.

Run both binary searches.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search Bounds
Trigger: identify the signal in the question.
Memory sentence: Two boundary searches isolate the equal-value range.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 48 • Koko Eating Bananas

MEDIUM | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find minimum eating speed to finish piles within h hours.

2. CORE OBSERVATION

Observation

For a fixed speed, required hours are monotonic: faster never needs more hours.

STATE

INVARIANT

TRANSITION

lo/hi speed and feasibility predicate.

All speeds $\geq$ a feasible speed are feasible.

Binary search speed; feasible if $\text{sum}(\text{ceil}(\text{pile}/\text{speed})) \leq h$.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search on Answer

Trigger: identify the signal in the question.

Memory sentence: For a fixed speed, required hours are monotonic: faster never needs more hours.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 49 • Capacity To Ship Packages Within D Days

MEDIUM | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find minimum ship capacity to deliver packages in order within D days.

2. CORE OBSERVATION

Observation

Feasibility is monotonic with capacity.

STATE

INVARIANT

TRANSITION

lo=max weight, hi=sum weights.

For a capacity, greedy loading uses the minimum possible days.

Binary search capacity; simulate days greedily.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search on Answer
Trigger: identify the signal in the question.
Memory sentence: Feasibility is monotonic with capacity.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 50 • Min Speed to Arrive on Time

MEDIUM | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find minimum integer speed to reach all train destinations within hour.

2. CORE OBSERVATION

Observation

Travel time decreases monotonically as speed increases.

STATE

INVARIANT

TRANSITION

lo/hi speed.

All speeds $\geq$ a feasible speed remain feasible.

Binary search speed; sum ceil for all but last leg and exact last-leg time.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search on Answer
Trigger: identify the signal in the question.
Memory sentence: Travel time decreases monotonically as speed increases.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 51 • Aggressive cows

MEDIUM | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Place cows in stalls maximizing minimum pairwise distance.

2. CORE OBSERVATION

Observation

For a proposed distance d , greedy placement determines feasibility; feasibility is monotonic.

STATE

INVARIANT

TRANSITION

lo/hi distance.

If d is feasible, every smaller distance is feasible.

Binary search d ; greedily place cows at earliest possible stalls.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Binary Search on Answer

Trigger: identify the signal in the question.

Memory sentence: For a proposed distance d , greedy placement determines feasibility; feasibility is monotonic.

COMPLEXITY

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 52 • Min number of days to make m bouquets

MEDIUM | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find earliest day when m bouquets of k adjacent flowers can be made.

2. CORE OBSERVATION

Observation

For a given day, feasibility is monotonic.

STATE

INVARIANT

TRANSITION

lo/hi day and feasible predicate.

If bouquets are possible on day d, they remain possible on later days.

Binary search day; scan flowers and count groups of k bloomed flowers.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search on Answer
Trigger: identify the signal in the question.
Memory sentence: For a given day, feasibility is monotonic.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 53 • Magnetic Force Between Two Balls

MEDIUM | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Maximize minimum distance between placed balls.
--------	---

2. CORE OBSERVATION

Observation	For a candidate distance, greedy placement is feasible or not monotonically.
-------------	--

STATE	INVARIANT	TRANSITION
lo/hi distance.	If distance d works, any smaller distance works.	Binary search d; place each ball at earliest position at least d away.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL

Pattern: Binary Search on Answer
Trigger: identify the signal in the question.
Memory sentence: For a candidate distance, greedy placement is feasible or not monotonically.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 54 • Allocate Minimum Number of Pages

MEDIUM | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Split contiguous books among students minimizing the maximum pages assigned.
--------	--

2. CORE OBSERVATION

Observation	For a proposed maximum pages, greedy allocation tells whether students suffice; feasibility is monotonic.
-------------	---

STATE	INVARIANT	TRANSITION
lo=max pages, hi=total pages.	Each capacity gives the minimum number of students needed by greedy partitioning.	Binary search capacity and check $\text{studentsNeeded} \leq m$.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL	COMPLEXITY
Pattern: Binary Search on Answer Trigger: identify the signal in the question. Memory sentence: For a proposed maximum pages, greedy allocation tells whether students suffice; feasibility is monotonic.	Time: _____ Space: _____
REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐	Confidence: 1 2 3 4 5

PROBLEM 55 • Split Array Largest Sum

HARD | Binary Search on Answer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Split array into at most/ exactly m contiguous parts minimizing the largest part sum.

2. CORE OBSERVATION

Observation

Same monotonic capacity predicate as book allocation.

STATE

INVARIANT

TRANSITION

lo=max element, hi=total sum.

For capacity C, greedy creates the fewest segments whose sums $\leq C$.

Binary search C and keep the smallest feasible value.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search on Answer
Trigger: identify the signal in the question.
Memory sentence: Same monotonic capacity predicate as book allocation.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 56 • Search a 2D Matrix

MEDIUM | Binary Search

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?		
Answer	Search a target in a matrix with row-major sorted order.	
2. CORE OBSERVATION		
Observation	Treat the matrix as one sorted 1D array.	
STATE	INVARIANT	TRANSITION
l/r over 0..rows*cols-1.	The target, if present, stays in the candidate interval.	Map mid to row=mid/cols, col=mid%cols and binary search.
Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.	
INTERVIEW QUICK RECALL		COMPLEXITY
Pattern: Binary Search Trigger: identify the signal in the question. Memory sentence: Treat the matrix as one sorted 1D array.		Time: _____ Space: _____
REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐		Confidence: 1 2 3 4 5

PROBLEM 57 • Search a 2D Matrix II

MEDIUM | Matrix Staircase Search

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Search target in matrix sorted left-to-right and top-to-bottom.

2. CORE OBSERVATION

Observation

Start at top-right: one comparison eliminates a whole row or column.

STATE

INVARIANT

TRANSITION

$r=0, c=\text{lastCol}$.

Remaining candidate region is the staircase below/left of current cell.

If $\text{value} > \text{target}$ move left; if $\text{value} < \text{target}$ move down; else found.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Matrix Staircase Search
Trigger: identify the signal in the question.
Memory sentence: Start at top-right: one comparison eliminates a whole row or column.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 58 • Kth Smallest Element in Sorted Matrix

MEDIUM | Binary Search on Value

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find kth smallest in a row/column sorted matrix.

2. CORE OBSERVATION

Observation

Count how many values $\leq x$; that count is monotonic in x .

STATE

INVARIANT

TRANSITION

lo=min matrix value, hi=max value.

For a candidate x , count(x) is the number of elements $\leq x$.

Binary search x ; if count $\geq k$ move left, else right.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search on Value
Trigger: identify the signal in the question.
Memory sentence: Count how many values $\leq x$; that count is monotonic in x .

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 59 • Matrix Median

HARD | Binary Search on Value

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find median of a row-wise sorted matrix.

2. CORE OBSERVATION

Observation

Count elements $\leq x$ across rows; binary search the value whose rank crosses half.

STATE

INVARIANT

TRANSITION

lo=min row first, hi=max row last.

count(x) is monotonic; median is smallest x with enough elements $\leq x$.

Binary search value and use upper bounds per row.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Binary Search on Value
Trigger: identify the signal in the question.
Memory sentence: Count elements $\leq x$ across rows; binary search the value whose rank crosses half.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 60 • Next Greater Element I

EASY | Monotonic Stack

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

For each element, find the first greater element to its right.

2. CORE OBSERVATION

Observation

A decreasing monotonic stack stores unresolved elements.

STATE

Stack holds values whose next greater element is not found.

INVARIANT

Stack is decreasing from bottom to top; when a larger x arrives it resolves all smaller tops.

TRANSITION

Pop while $\text{top} < x$ and map popped $\rightarrow x$; then push x .

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Monotonic Stack
Trigger: identify the signal in the question.
Memory sentence: A decreasing monotonic stack stores unresolved elements.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 61 • Next Greater Element II

MEDIUM | Monotonic Stack / Circular

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find next greater element in a circular array.

2. CORE OBSERVATION

Observation

Traverse conceptually twice; unresolved indices use a monotonic decreasing stack.

STATE

Stack stores indices from the virtual doubled traversal.

INVARIANT

Stack values are decreasing and unresolved; second pass can resolve first-pass elements.

TRANSITION

Loop $i=0..2n-1$; $x=nums[i\%n]$; pop smaller and assign; push i only in first pass.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Monotonic Stack / Circular
Trigger: identify the signal in the question.
Memory sentence: Traverse conceptually twice; unresolved indices use a monotonic decreasing stack.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 62 • Daily Temperatures

MEDIUM | Monotonic Stack

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

For each day, find days until a warmer temperature.

2. CORE OBSERVATION

Observation

Next greater element on indices.

STATE

INVARIANT

TRANSITION

Stack stores indices of decreasing temperatures.

For unresolved indices in stack, no warmer day has appeared yet.

While current temp > stack top temp, pop idx and set answer[idx] = i - idx; push i.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Monotonic Stack
Trigger: identify the signal in the question.
Memory sentence: Next greater element on indices.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 63 • Online Stock Span

MEDIUM | Monotonic Stack

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

For each price, return consecutive previous days with price $\leq$ today.

2. CORE OBSERVATION

Observation

Collapse dominated previous prices using a monotonic decreasing stack.

STATE

INVARIANT

TRANSITION

Stack stores [price,span].

Each stack item represents a block of consecutive days with prices $\leq$ its price.

Pop while top.price $\leq$ price and add top.span; push combined span.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Monotonic Stack

Trigger: identify the signal in the question.

Memory sentence: Collapse dominated previous prices using a monotonic decreasing stack.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 64 • Asteroid Collision

MEDIUM | Stack Simulation

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Simulate collisions between moving asteroids.

2. CORE OBSERVATION

Observation

Only a positive asteroid on stack can collide with a new negative asteroid.

STATE

Stack is the surviving sequence so far.

INVARIANT

Stack contains exactly the asteroids that survive all processed collisions.

TRANSITION

While $top > 0$ and $current < 0$, compare magnitudes; pop smaller, discard equal, or discard current.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Stack Simulation

Trigger: identify the signal in the question.

Memory sentence: Only a positive asteroid on stack can collide with a new negative asteroid.

COMPLEXITY

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 65 • Largest Rectangle in Histogram

HARD | Monotonic Stack

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find the largest rectangle under histogram bars.

2. CORE OBSERVATION

Observation

For each bar, determine its maximal width where it is the minimum; a monotonic increasing stack finds boundaries.

STATE

INVARIANT

TRANSITION

Stack stores indices of increasing heights.

For each popped bar, current i is its first smaller right boundary and new stack top is first smaller left boundary.

When height decreases, pop and compute $\text{height} * (i - \text{left} - 1)$; flush stack at end.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Monotonic Stack

Trigger: identify the signal in the question.

Memory sentence: For each bar, determine its maximal width where it is the minimum; a monotonic increasing stack finds boundaries.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 66 • Maximal Rectangle

HARD | Monotonic Stack + Histogram

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find largest all-1 rectangle in a binary matrix.

2. CORE OBSERVATION

Observation

Convert each row into histogram heights and solve Largest Rectangle in Histogram.

STATE

heights[j] is consecutive 1s ending at current row.

INVARIANT

After processing a row, heights represent exactly the histogram for that row.

TRANSITION

Update heights row-by-row and run histogram max.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Monotonic Stack + Histogram
Trigger: identify the signal in the question.
Memory sentence: Convert each row into histogram heights and solve Largest Rectangle in Histogram.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 67 • Basic Calculator II

MEDIUM | Stack + Expression Evaluation

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Evaluate $+$, $-$, $*$, $/$ expression without parentheses.

2. CORE OBSERVATION

Observation

Multiplication/division must be resolved before addition/subtraction; a stack stores signed terms.

STATE

INVARIANT

TRANSITION

stack of terms and current number/operator.

Stack contains terms whose sum is the value of processed prefix with precedence resolved.

On operator, apply previous operator to current number; push result for $+$ / $-$ and combined result for $*$ / $/$; sum stack.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack + Expression Evaluation
Trigger: identify the signal in the question.
Memory sentence: Multiplication/division must be resolved before addition/subtraction; a stack stores signed terms.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 68 • Evaluate Reverse Polish Notation

MEDIUM | Stack / Postfix

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Evaluate postfix expression.

2. CORE OBSERVATION

Observation

Every operator combines the two most recent operands, naturally handled by a stack.

STATE

INVARIANT

TRANSITION

Operand stack.

Stack contains values of processed tokens not yet consumed by an operator.

Push numbers; on operator pop b then a, compute a op b, push result.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack / Postfix

Trigger: identify the signal in the question.

Memory sentence: Every operator combines the two most recent operands, naturally handled by a stack.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 69 • Decode String

MEDIUM | Stack Simulation

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer Decode nested k[encoded] strings.

2. CORE OBSERVATION

Observation Use stacks to remember the string and repeat count before each nested bracket.

STATE

count stack + string stack/current builder.

INVARIANT

At each nesting level, stored prefix and count represent the state before entering the bracket.

TRANSITION

On digit build count; [push state;] pop and combine repeated current into previous.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Stack Simulation

Trigger: identify the signal in the question.

Memory sentence: Use stacks to remember the string and repeat count before each nested bracket.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 70 • Basic Calculator I

HARD | Stack + Expression Parsing

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Evaluate expression with +,- and parentheses.
--------	---

2. CORE OBSERVATION

Observation	A sign and stack can save the outer expression state when entering parentheses.
-------------	---

STATE	INVARIANT	TRANSITION
current result, number, sign, stack of previous result/sign.	Inside each parenthesis, result represents the value of the processed prefix under current sign context.	On '(' push result/sign and reset; on ')' combine inner result with saved sign/result.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL	COMPLEXITY
Pattern: Stack + Expression Parsing Trigger: identify the signal in the question. Memory sentence: A sign and stack can save the outer expression state when entering parentheses.	Time: _____ Space: _____
REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐	Confidence: 1 2 3 4 5

PROBLEM 71 • Backspace String Compare

EASY | Stack Simulation / Two Pointer

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Check whether two strings are equal after processing # as backspace.

2. CORE OBSERVATION

Observation

Backspaces can be simulated by stack or by scanning backward with skip counts.

STATE

For backward scan: pointer and skip count per string.

INVARIANT

At each pointer, skip count exactly equals pending backspaces affecting preceding characters.

TRANSITION

Consume # by incrementing skip; skip characters when skip>0; compare next live chars.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Stack Simulation / Two Pointer
Trigger: identify the signal in the question.
Memory sentence: Backspaces can be simulated by stack or by scanning backward with skip counts.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 72 • Remove All Adjacent Duplicates

EASY | Stack Simulation

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Repeatedly remove adjacent equal pairs until stable.

2. CORE OBSERVATION

Observation

A stack makes the last surviving character immediately available for cancellation.

STATE

INVARIANT

TRANSITION

Stack of surviving characters.

Stack equals the fully reduced form of the processed prefix.

If top==current pop; else push.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack Simulation
Trigger: identify the signal in the question.
Memory sentence: A stack makes the last surviving character immediately available for cancellation.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 73 • Make the String Great

EASY | Stack Simulation

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Remove adjacent letters that are the same alphabetic character with opposite case.

2. CORE OBSERVATION

Observation

The conflict is local and the stack exposes the last surviving character.

STATE

INVARIANT

TRANSITION

Stack of surviving chars.

Stack is the reduced good string for processed prefix.

If top differs only by case, pop; else push.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack Simulation
Trigger: identify the signal in the question.
Memory sentence: The conflict is local and the stack exposes the last surviving character.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 74 • Minimum String Length After Removing Substrings

MEDIUM | Stack Simulation

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Repeatedly remove specified adjacent substrings and return minimum length.
--------	--

2. CORE OBSERVATION

Observation	Use a stack to expose newly formed suffixes after every removal.
-------------	--

STATE	INVARIANT	TRANSITION
Stack of surviving characters / suffix.	Stack represents the fully reduced prefix under the allowed deletions.	Append chars; whenever suffix matches a removable pattern, delete it.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL

Pattern: Stack Simulation
Trigger: identify the signal in the question.
Memory sentence: Use a stack to expose newly formed suffixes after every removal.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 75 • Valid Parentheses

EASY | Stack / Parentheses

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Check whether brackets are correctly nested and matched.

2. CORE OBSERVATION

Observation

Opening brackets must be closed in reverse order, exactly LIFO.

STATE

INVARIANT

TRANSITION

Stack of opening brackets.

Stack contains unmatched openings for processed prefix; top must match any closing bracket.

Push openings; on closing verify top match then pop; stack must end empty.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack / Parentheses

Trigger: identify the signal in the question.

Memory sentence: Opening brackets must be closed in reverse order, exactly LIFO.

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 76 • Minimum Add to Make Parentheses Valid

MEDIUM | Balance Counter

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find minimum insertions needed to make parentheses balanced.

2. CORE OBSERVATION

Observation

Track unmatched opens and unmatched closes; each contributes one insertion.

STATE

balance = unmatched opens; additions = unmatched closes.

INVARIANT

balance never goes negative; every negative close requires one inserted opening.

TRANSITION

For an open, balance++; for a close, if balance>0 decrement else additions++; answer additions+balance.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Balance Counter

Trigger: identify the signal in the question.

Memory sentence: Track unmatched opens and unmatched closes; each contributes one insertion.

COMPLEXITY

Time: _____

Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 77 • Score of Parentheses

MEDIUM | Stack / Nested Structure

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Compute score where $()=1$, concatenation adds, nesting doubles.

2. CORE OBSERVATION

Observation

A stack stores the score at each nesting level.

STATE

INVARIANT

TRANSITION

Stack of scores; current frame score.

Current frame accumulates scores of complete groups inside it.

On open push current frame; reset; on close pop outer and add $outer + 2*inner$.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack / Nested Structure
Trigger: identify the signal in the question.
Memory sentence: A stack stores the score at each nesting level.

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 78 • Longest Valid Parentheses

HARD | Stack / Boundary Tracking

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Find longest contiguous valid-parentheses substring.

2. CORE OBSERVATION

Observation

A stack of indices marks the last unmatched boundary; valid length is i-boundary.

STATE

Stack contains indices of unmatched opens plus a sentinel boundary.

INVARIANT

Top is the index immediately before the current valid region.

TRANSITION

On open push i; on close pop; if empty push i as new invalid boundary; else update i-top.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Stack / Boundary Tracking
Trigger: identify the signal in the question.
Memory sentence: A stack of indices marks the last unmatched boundary; valid length is i-boundary.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time: _____
Space: _____

Confidence: 1 2 3 4 5

PROBLEM 79 • Implement Queue using Stacks

EASY | Stack-Based Design

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Implement FIFO queue operations using LIFO stacks.
--------	--

2. CORE OBSERVATION

Observation	Use an input stack for enqueue and an output stack for dequeue; transfer only when output is empty.
-------------	---

STATE	INVARIANT	TRANSITION
inStack/outStack.	When outStack is nonempty, its top is always the oldest queued element.	Push into in; for pop/peek, if out empty transfer all in->out, then operate on out.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL

Pattern: Stack-Based Design
Trigger: identify the signal in the question.
Memory sentence: Use an input stack for enqueue and an output stack for dequeue; transfer only when output is empty.

COMPLEXITY

Time: _____
Space: _____

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

PROBLEM 80 • Implement Stack using Queues

EASY | Stack-Based Design

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Implement a LIFO stack using FIFO queue operations. Support push, pop, top, and empty.
--------	--

2. CORE OBSERVATION

Observation	Make the newest element appear at the queue end used as the logical stack top.
-------------	--

STATE	INVARIANT	TRANSITION
Queue order + logical top.	The queue order always represents the stack in LIFO order.	On push, rotate/rearrange so the new element reaches the logical top; pop/top use that end.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL	COMPLEXITY
Pattern: Stack-Based Design Trigger: identify the signal in the question. Memory sentence: Make the newest element appear at the queue end used as the logical stack top.	Time/Space: $O(n)$ push, $O(1)$ pop/top with one-queue rotation; $O(n)$ space.
REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐	Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 81 • Min Stack

MEDIUM | Stack-Based Design

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Design a stack that supports push, pop, top, and retrieving the minimum element in $O(1)$.

2. CORE OBSERVATION

Observation

Store enough minimum information so a pop can restore the previous minimum immediately.

STATE

INVARIANT

TRANSITION

Main stack + current-min information.

Stored minimum equals the minimum of all elements currently in the stack.

Push updates min; pop restores the previous min; top reads without scanning.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack-Based Design
Trigger: identify the signal in the question.
Memory sentence: Store enough minimum information so a pop can restore the previous minimum immediately.

Time/Space: $O(1)$ per operation, $O(n)$ space.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 82 • Max Stack

MEDIUM | Stack-Based Design

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Design a stack that supports normal stack operations plus retrieving/removing the maximum element efficiently.

2. CORE OBSERVATION

Observation

LIFO access and maximum access are different requirements, so maintain synchronized auxiliary information.

STATE

Stack + auxiliary ordered/max structure.

INVARIANT

The auxiliary structure stays synchronized with the actual stack contents and identifies the current maximum.

TRANSITION

Update both structures on push/pop; use the auxiliary structure for maximum operations.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Stack-Based Design
Trigger: identify the signal in the question.
Memory sentence: LIFO access and maximum access are different requirements, so maintain synchronized auxiliary information.

COMPLEXITY

Time/Space: Depends on implementation; typically $O(\log n)$ or $O(n)$ for removeMax and $O(1)$ stack operations.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 83 • Design Stack with Increment Operation

MEDIUM | Stack-Based Design

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Implement a bounded stack with push, pop, and increment(k,val) for the bottom k elements.

2. CORE OBSERVATION

Observation

Do not immediately update all k elements; store a lazy increment and propagate it when elements are popped.

STATE

Stack values + lazy increment metadata.

INVARIANT

Pending increment at an index represents work still owed to affected lower elements.

TRANSITION

Record the increment at the affected boundary; apply it during pop and pass the pending amount downward.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Stack-Based Design

Trigger: identify the signal in the question.

Memory sentence: Do not immediately update all k elements; store a lazy increment and propagate it when elements are popped.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time/Space: O(1) per operation, O(n) space.

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 84 • Remove K Digits

MEDIUM | Stack + Greedy

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Remove exactly k digits so the remaining number is as small as possible.

2. CORE OBSERVATION

Observation

An earlier larger digit is more harmful than a later smaller digit, so remove it whenever a smaller next digit appears.

STATE

INVARIANT

TRANSITION

Monotonic increasing stack + remaining k.

The stack is the smallest feasible prefix after processing the current digits.

While $k > 0$ and $top > current$, pop; push current; if k remains, remove from the end.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack + Greedy
Trigger: identify the signal in the question.
Memory sentence: An earlier larger digit is more harmful than a later smaller digit, so remove it whenever a smaller next digit appears.

Time/Space: $O(n)$ time, $O(n)$ space.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 85 • Remove Duplicate Letters

MEDIUM | Stack + Greedy

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Return the lexicographically smallest string containing every distinct character exactly once.

2. CORE OBSERVATION

Observation

A larger character can be removed only if it appears again later.

STATE

INVARIANT

TRANSITION

Monotonic stack + seen set + remaining frequency.

The stack is the lexicographically smallest valid distinct-character prefix.

Pop larger reusable characters; skip already-used characters; push the current character when selected.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Stack + Greedy

Trigger: identify the signal in the question.

Memory sentence: A larger character can be removed only if it appears again later.

Time/Space: $O(n)$ time, $O(n)$ space.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 86 • Smallest Subsequence of Distinct Characters

MEDIUM | Stack + Greedy

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Build the lexicographically smallest subsequence containing each distinct character exactly once.

2. CORE OBSERVATION

Observation

Keep the smallest feasible prefix and pop a larger character only when it can appear again later.

STATE

Monotonic stack + used set + remaining frequency.

INVARIANT

The stack is the smallest valid distinct-character prefix for the processed input.

TRANSITION

Pop larger reusable characters; skip used characters; push the current character when first selected.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Stack + Greedy
Trigger: identify the signal in the question.
Memory sentence: Keep the smallest feasible prefix and pop a larger character only when it can appear again later.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time/Space: $O(n)$ time, $O(n)$ space.

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 87 • Minimum Remove to Make Valid Parentheses

MEDIUM | Stack + Greedy

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Remove the fewest parentheses so the remaining string is valid.
--------	---

2. CORE OBSERVATION

Observation	A closing parenthesis without an unmatched opening is invalid; leftover openings are invalid at the end.
-------------	--

STATE	INVARIANT	TRANSITION
Indices of unmatched opening parentheses + invalid closing positions.	After a prefix, the stack contains exactly the unmatched '(' indices.	Push '('; match ')' by popping when possible; otherwise mark it invalid; remove leftover opens.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL

Pattern: Stack + Greedy
Trigger: identify the signal in the question.
Memory sentence: A closing parenthesis without an unmatched opening is invalid; leftover openings are invalid at the end.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

COMPLEXITY

Time/Space: $O(n)$ time, $O(n)$ space.

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 88 • Create Maximum Number

HARD | Stack + Greedy

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Choose digits from two arrays, preserving order, to form the largest number of a required length.

2. CORE OBSERVATION

Observation

Choose the best subsequence from each array, then merge the candidates lexicographically.

STATE

Maximum subsequences + merge position.

INVARIANT

Each selected subsequence is the lexicographically largest possible subsequence of its required length.

TRANSITION

Try every valid split; build each best subsequence; merge by comparing suffixes; keep the best result.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

Pattern: Stack + Greedy
Trigger: identify the signal in the question.
Memory sentence: Choose the best subsequence from each array, then merge the candidates lexicographically.

COMPLEXITY

Time/Space: For n, m total lengths, the exact bound depends on implementation; typically $O(k(n+m) \cdot \text{comparison cost})$.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 89 • Delete Middle Element of Stack

EASY | Recursive Stack

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer	Delete the middle element of a stack while preserving the order of all other elements.
--------	--

2. CORE OBSERVATION

Observation	Recursion can temporarily remove top elements and rebuild the stack while unwinding.
-------------	--

STATE	INVARIANT	TRANSITION
Original stack + recursion call stack.	Temporarily removed elements are waiting to be restored in reverse order.	Pop until the middle; skip it; push all other saved elements back during unwinding.

Correctness check	At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.
-------------------	---

INTERVIEW QUICK RECALL

Pattern: Recursive Stack
Trigger: identify the signal in the question.
Memory sentence: Recursion can temporarily remove top elements and rebuild the stack while unwinding.

COMPLEXITY

Time/Space: $O(n)$ time, $O(n)$ recursion space.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 90 • Reverse a Stack (Recursive)

MEDIUM | Recursive Stack

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Reverse a stack using recursion without an explicit auxiliary stack.

2. CORE OBSERVATION

Observation

Pop recursively to empty, then insert each saved item at the bottom during unwinding.

STATE

INVARIANT

TRANSITION

Original stack + recursion call stack.

The smaller stack is already reversed before the saved element is inserted at its bottom.

Pop top; reverse the remainder; insert the saved top at the bottom.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Recursive Stack
Trigger: identify the signal in the question.
Memory sentence: Pop recursively to empty, then insert each saved item at the bottom during unwinding.

Time/Space: $O(n^2)$ time with a recursive insert-at-bottom helper, $O(n)$ recursion space.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework

PROBLEM 91 • Insert at Bottom of Stack

MEDIUM | Recursive Stack

1. WHAT PROBLEM DOES THIS PATTERN SOLVE?

Answer

Insert an element at the bottom of a stack using recursion.

2. CORE OBSERVATION

Observation

Remove the top recursively until the stack is empty, insert the new item, then restore the removed elements.

STATE

INVARIANT

TRANSITION

Original stack + recursion call stack.

All temporarily removed elements return in their original order above the new bottom.

If empty, push x; otherwise pop top, recurse, then push the saved top back.

Correctness check

At every iteration, the invariant must remain true. If you can state what the variables mean before coding, the transition is usually the code.

INTERVIEW QUICK RECALL

COMPLEXITY

Pattern: Recursive Stack
Trigger: identify the signal in the question.
Memory sentence: Remove the top recursively until the stack is empty, insert the new item, then restore the removed elements.

Time/Space: $O(n)$ time, $O(n)$ recursion space.

REVISION TRACK +3 ☐ +7 ☐ +14 ☐ +30 ☐ +60 ☐

Confidence: 1 2 3 4 5

CODE RECONSTRUCTION — write from memory after recalling the framework